

BAB III

LANDASAN TEORI

3.1 Konsep Sistem Informasi

Menurut McLeod dan Schell (2007), sistem informasi adalah kombinasi dari teknologi informasi dan aktivitas orang yang menggunakan teknologi itu untuk mendukung operasi dan manajemen. Sistem informasi tidak hanya mencakup teknologi informasi (hardware, software, teknologi database, dan teknologi telekomunikasi), tetapi juga aktivitas manusia yang memproses informasi tersebut.

O'Brien dan Marakas (2011) mendefinisikan sistem informasi sebagai kombinasi teratur dari orang-orang, hardware, software, jaringan komunikasi, dan sumber daya data yang mengumpulkan, menyaring, memproses, menciptakan, dan mendistribusikan data tentang suatu organisasi. Definisi ini menekankan bahwa sistem informasi bukan hanya tentang teknologi, tetapi juga tentang bagaimana manusia berinteraksi dengan teknologi tersebut.

Laudon dan Laudon (2014) menjelaskan bahwa sistem informasi dapat didefinisikan secara teknis sebagai sekumpulan komponen yang saling berhubungan yang mengumpulkan, memproses, menyimpan, dan mendistribusikan informasi untuk mendukung pengambilan keputusan dan kendali dalam suatu organisasi.

3.1.1 Komponen Sistem Informasi

Menurut Whitten dan Bentley (2007), sistem informasi terdiri dari lima komponen utama, yaitu :

- 1) *Hardware*

Komponen fisik dari teknologi informasi, termasuk komputer, printer, dan perangkat jaringan.

2) *Software*

Program dan instruksi yang memberitahu *hardware* apa yang harus dilakukan.

3) *Data*

Fakta mentah yang diorganisir dan diproses untuk menghasilkan informasi.

4) *Network*

Sistem komunikasi yang memungkinkan berbagi sumber daya dan informasi.

5) *People*

Sumber daya manusia yang mengelola, menjalankan, memprogram, dan memelihara sistem.

3.1.2 Manfaat Sistem Informasi

Hall (2011) menyebutkan beberapa manfaat utama sistem informasi bagi organisasi :

1) Efisiensi Operasional

Mengotomatisasi proses bisnis dan mengurangi waktu pemrosesan

2) Peningkatan Kualitas Informasi

Memberikan informasi yang akurat, tepat waktu, dan relevan

3) Dukungan Pengambilan Keputusan

Menyediakan informasi yang diperlukan untuk pengambilan keputusan yang efektif

4) Peningkatan Komunikasi

Memfasilitasi komunikasi internal dan eksternal organisasi

3.2 Sistem Informasi Berbasis *Web*

Menurut Turban et al. (2015), sistem informasi berbasis *web* adalah sistem informasi yang menggunakan teknologi *web* sebagai platform untuk mengakses, memproses, dan menyajikan informasi. Sistem ini memungkinkan pengguna untuk mengakses aplikasi melalui *web browser* dari berbagai lokasi dan perangkat.

Pressman (2010) menjelaskan bahwa aplikasi *web* adalah sistem perangkat lunak yang dirancang untuk berjalan di atas infrastruktur *World Wide Web*. Aplikasi *web* menggabungkan berbagai teknologi termasuk HTML, CSS, *JavaScript* di sisi klien, serta teknologi *server-side* seperti PHP, ASP.NET, atau *framework* lainnya.

3.2.1 Karakteristik Sistem Berbasis Web

Menurut Ginige dan Murugesan (2001), sistem berbasis *web* memiliki karakteristik khusus :

1) *Platform Independence*

Dapat diakses dari berbagai sistem operasi dan perangkat

2) *Network-centric*

Bergantung pada jaringan untuk fungsinya

3) *Multi-user*

Mendukung akses simultan dari multiple users

4) *Dynamic Content*

Konten dapat berubah secara dinamis berdasarkan interaksi pengguna

5) *Scalability*

Dapat disesuaikan dengan pertumbuhan jumlah pengguna

3.2.2 Keunggulan Sistem Berbasis *Web*

Sommerville (2011) mengidentifikasi beberapa keunggulan sistem berbasis *web* diantaranya :

1) Kemudahan Akses

Dapat diakses dari mana saja selama terdapat koneksi internet, sistem berbasis *web* memberikan fleksibilitas tinggi kepada pengguna untuk mengakses layanan atau informasi kapan pun dan di mana pun. Hal ini sangat mendukung mobilitas kerja, terutama dalam lingkungan kerja modern yang semakin mengandalkan fleksibilitas lokasi dan waktu.

2) *Cross-platform Compatibility*

Karena berjalan melalui *browser web*, sistem berbasis *web* tidak tergantung pada sistem operasi tertentu. Baik pengguna menggunakan *Windows*, *macOS*, *Linux*, maupun sistem operasi *mobile* seperti *Android* dan *iOS*, mereka tetap dapat mengakses sistem dengan pengalaman yang konsisten. Ini mengurangi kebutuhan untuk pengembangan perangkat lunak yang terpisah untuk setiap platform.

3) Kemudahan *Maintenance*

Pembaruan sistem hanya perlu dilakukan pada sisi *server*, tanpa perlu menginstal atau memperbarui perangkat lunak pada setiap perangkat pengguna. Hal ini menyederhanakan proses pemeliharaan dan memastikan bahwa semua pengguna selalu menggunakan versi terbaru dari aplikasi, sehingga mengurangi potensi *bug* atau inkonsistensi sistem.

4) *Cost-effective*

Sistem berbasis *web* dapat mengurangi biaya distribusi dan instalasi karena tidak memerlukan media fisik atau prosedur instalasi yang kompleks di sisi pengguna. Selain itu, pengelolaan dan pembaruan terpusat di *server* juga mengurangi beban kerja tim teknis, yang pada akhirnya menurunkan total biaya operasional sistem dalam jangka panjang.

3.3 *Framework Laravel*

Laravel adalah *framework* PHP yang dikembangkan oleh Taylor Otwell pada tahun 2011. Menurut Stauffer (2019), *Laravel* adalah *framework web application* yang menggunakan sintaks yang ekspresif dan elegan, dirancang untuk membuat pengembangan *web* menjadi lebih menyenangkan dan produktif dengan menyederhanakan tugas-tugas umum yang digunakan dalam sebagian besar proyek *web*.

Otwell (2020) menjelaskan bahwa *Laravel* dibangun dengan filosofi "*convention over configuration*", yang berarti *framework* menyediakan struktur dan konvensi yang masuk akal sehingga *developer* dapat fokus pada logika aplikasi daripada konfigurasi yang repetitif.

3.3.1 Fitur Utama *Laravel*

Menurut dokumentasi resmi *Laravel* (*Laravel Team*, 2023), *framework* ini menyediakan beberapa fitur unggulan, yaitu :

1) *Eloquent ORM*

Object-Relational Mapping (ORM) ini memungkinkan pengembang untuk berinteraksi dengan *database* menggunakan sintaks PHP yang ekspresif dan berorientasi objek, tanpa harus menulis kueri SQL secara langsung. *Eloquent* menyederhanakan proses manipulasi data,

seperti membuat, membaca, memperbarui, dan menghapus (CRUD), serta mendukung relasi antar tabel seperti *one-to-many*, *many-to-many*, dan lain-lain dengan cara yang intuitif.

2) *Blade Templating Engine*

Blade adalah sistem *templating* bawaan *Laravel* yang ringan namun *powerful*. Dengan *Blade*, pengembang dapat membuat tampilan antarmuka yang dinamis menggunakan sintaks yang bersih dan mudah dibaca. *Blade* juga mendukung fitur seperti pewarisan *layout* (*template inheritance*), perulangan, pengkondisian, dan komponen, yang memudahkan dalam memisahkan logika dan tampilan aplikasi.

3) *Artisan Command Line Interface*

Artisan adalah antarmuka baris perintah (CLI) milik *Laravel* yang menyediakan berbagai perintah untuk mempercepat proses pengembangan. Dengan *Artisan*, pengembang dapat membuat *controller*, *model*, *migration*, *seeder*, dan masih banyak lagi secara otomatis. Fitur ini meningkatkan produktivitas karena mengurangi kebutuhan menulis kode dasar yang berulang secara manual.

4) *MVC Architecture*

Laravel mengikuti pola arsitektur *Model-View-Controller*, yang memisahkan logika aplikasi, antarmuka pengguna, dan manajemen data menjadi bagian-bagian terpisah. Hal ini membuat kode menjadi lebih terorganisir, mudah dikelola, dan mendukung pengembangan kolaboratif. MVC juga meningkatkan skalabilitas dan pemeliharaan aplikasi dalam jangka panjang.

5) *Route System*

Sistem *routing* di *Laravel* sangat fleksibel dan ekspresif, memungkinkan pengembang untuk mendefinisikan jalur URL dan metode HTTP dengan cara yang mudah dibaca. Selain mendukung route dengan parameter dan grup *route*, *Laravel* juga menyediakan fitur seperti *route naming*, *route model binding*, dan *middleware routing* untuk mengatur logika akses dengan lebih efisien.

6) *Middleware*

Middleware berfungsi sebagai lapisan perantara antara *request* HTTP dan aplikasi, yang memungkinkan pengembang menyisipkan logika seperti autentikasi, *logging*, atau validasi sebelum request diteruskan ke *controller*. Dengan konsep ini, proses penyaringan dan pengamanan request menjadi lebih modular dan mudah dipelihara.

7) *Authentication System*

Laravel dilengkapi dengan sistem autentikasi yang lengkap dan aman, mencakup fitur login, registrasi, manajemen sesi, dan otorisasi pengguna. Selain itu, *Laravel* juga menyediakan paket tambahan seperti *Laravel Breeze*, *Jetstream*, dan *Fortify* yang menyederhanakan implementasi autentikasi modern termasuk verifikasi email, otentikasi dua faktor, dan integrasi dengan API token.

3.3.2 Arsitektur MVC Laravel

Menurut Pitt (2016), *Laravel* mengimplementasikan pola arsitektur *Model-View-Controller* (MVC) :

1) *Model*

Model bertanggung jawab untuk menangani seluruh logika yang berkaitan dengan data, termasuk pengambilan, penyimpanan, pembaruan, dan penghapusan data dari *database*. Selain itu, *Model* juga merepresentasikan struktur data dan relasinya, serta memvalidasi data sebelum dikirim ke atau dari basis data. Dalam kerangka kerja seperti *Laravel*, *Model* berperan sebagai jembatan langsung antara aplikasi dan *database*, menggunakan *Eloquent ORM* untuk mempermudah interaksi tersebut.

2) *View*

View berfungsi untuk menangani tampilan antarmuka pengguna (*user interface*) dan menyajikan data yang diterima dari *Controller* dalam format yang mudah dipahami. *View* tidak mengandung logika aplikasi, melainkan hanya fokus pada bagaimana data ditampilkan ke pengguna. Dengan menggunakan sistem *templating* seperti *Blade* di *Laravel*, *View* memungkinkan pengembang memisahkan antara logika bisnis dan elemen visual aplikasi secara jelas dan efisien.

3) *Controller*

Controller bertindak sebagai perantara antara *Model* dan *View*. Ia menerima input dari pengguna melalui *View*, memprosesnya sesuai logika aplikasi, dan kemudian berinteraksi dengan *Model* untuk mengambil atau memodifikasi data. Setelah itu, *Controller* mengatur bagaimana data tersebut dikirim kembali ke *View* untuk ditampilkan. Dengan kata lain, *Controller* mengoordinasikan alur kerja aplikasi dan menjadi pusat pengambilan keputusan dalam arsitektur MVC.

3.4 *Database Management System (DBMS)*

Menurut Elmasri dan Navathe (2016), *Database Management System* (DBMS) adalah perangkat lunak yang memungkinkan pengguna untuk mendefinisikan, membuat, memelihara, dan mengontrol akses ke *database*. DBMS berfungsi sebagai *interface* antara pengguna dengan *database* fisik.

Connolly dan Begg (2015) mendefinisikan DBMS sebagai sistem perangkat lunak yang memungkinkan pengguna untuk mendefinisikan, membuat, memelihara, dan mengontrol akses ke *database*, serta menyediakan cara untuk menyimpan, mengatur, dan mengambil data secara efisien.

3.4.1 *MySQL Database*

MySQL adalah sistem manajemen basis data relasional (RDBMS) *open source* yang dikembangkan oleh Oracle Corporation. Menurut DuBois (2013), *MySQL* adalah salah satu *database management system* yang paling populer di dunia, terutama untuk aplikasi *web*, karena keandalannya, kemudahan penggunaan, dan performa yang baik.

Welling dan Thomson (2017) menjelaskan bahwa *MySQL* memiliki beberapa keunggulan yaitu :

1) *Open Source*

MySQL merupakan perangkat lunak *open source* yang didistribusikan di bawah lisensi GNU *General Public License* (GPL), yang berarti pengguna bebas untuk mengunduh, menggunakan, memodifikasi, dan mendistribusikan perangkat lunak tersebut tanpa biaya. Ketersediaan kode sumber memungkinkan komunitas dan pengembang independen untuk terus berkontribusi dalam pengembangan dan peningkatan fitur, sehingga menjadikan *MySQL*

sebagai solusi *database* yang sangat fleksibel dan terus berkembang sesuai kebutuhan pengguna.

2) *Cross-platform*

MySQL mendukung berbagai platform utama seperti *Windows*, *Linux*, *Unix*, dan *macOS*, yang memungkinkan pengembang untuk mengembangkan dan mengelola aplikasi di berbagai lingkungan tanpa perlu khawatir tentang ketergantungan platform. Dukungan lintas platform ini menjadikan *MySQL* pilihan ideal untuk pengembangan perangkat lunak yang membutuhkan portabilitas dan fleksibilitas tinggi.

3) *High Performance*

MySQL dirancang untuk performa tinggi dengan penggunaan algoritma *indexing* yang efisien dan penyimpanan data yang teroptimasi. *Engine* seperti *InnoDB* dan *MyISAM* menawarkan berbagai opsi untuk meningkatkan performa *query*, baik untuk operasi baca maupun tulis. *MySQL* juga mendukung *caching query* dan replikasi, yang membantu dalam mengurangi waktu respons dan meningkatkan efisiensi penggunaan sumber daya.

4) *Scalability*

MySQL mampu menangani volume data yang besar dan beban transaksi yang tinggi, menjadikannya cocok untuk aplikasi skala kecil hingga *enterprise*. Dengan kemampuan *clustering* dan replikasi, *MySQL* memungkinkan penskalaan horizontal dan vertikal, yang penting dalam lingkungan yang terus tumbuh dan membutuhkan pengolahan data secara *real-time*.

5) *Security*

MySQL menyediakan fitur keamanan tingkat lanjut seperti otentikasi berbasis *host*, enkripsi koneksi menggunakan SSL, dan kontrol akses berbasis per pengguna. *MySQL* juga mendukung pengaturan hak akses yang granular terhadap *database*, tabel, dan kolom, sehingga *administrator* dapat memastikan bahwa hanya pengguna yang berwenang yang dapat mengakses data tertentu.

6) *Reliability*

MySQL telah terbukti sebagai sistem *database* yang andal dalam berbagai implementasi nyata, termasuk pada aplikasi web berskala besar dan sistem bisnis yang membutuhkan waktu aktif (*uptime*) tinggi. Fitur seperti *backup* otomatis, replikasi data, dan dukungan terhadap *recovery* setelah *crash* membantu menjamin integritas dan ketersediaan data, yang sangat penting dalam lingkungan produksi.

3.5 *Software Development Life Cycle (SDLC)*

Menurut Sommerville (2016), *Software Development Life Cycle (SDLC)* adalah proses terstruktur yang digunakan untuk merancang, mengembangkan, dan menguji *software* berkualitas baik. SDLC menyediakan *framework* sistematis untuk mengembangkan *software* dengan cara yang efisien dan terkontrol.

Pressman dan Maxim (2020) mendefinisikan SDLC sebagai metodologi yang digunakan untuk mengembangkan, memelihara, dan menggantikan sistem informasi dengan cara yang terstruktur dan sistematis.

3.5.1 Model Waterfall

Model *Waterfall* adalah salah satu model SDLC yang paling klasik dan banyak digunakan. Menurut Royce (1970), model *waterfall* adalah

pendekatan sekuensial untuk pengembangan software di mana progres mengalir secara bertahap dari atas ke bawah melalui fase-fase: *Requirements, Design, Implementation, Testing, Deployment, dan Maintenance.*

Boehm (1988) menjelaskan bahwa model *waterfall* memiliki karakteristik yaitu :

1) *Sequential Process*

Setiap fase harus diselesaikan sebelum melanjutkan ke fase berikutnya

2) *Documentation-driven*

Setiap fase menghasilkan *deliverable* yang terdokumentasi

3) *Predictable*

Timeline dan *milestone* dapat diprediksi dengan baik

4) *Quality Control*

Setiap fase memiliki *review gate* untuk *quality assurance*

3.5.2 Tahapan Model *Waterfall*

Menurut Bassil (2012), model *waterfall* terdiri dari enam tahapan utama yaitu :

1) *Requirements Analysis*

Mengumpulkan dan menganalisis kebutuhan sistem.

2) *System Design*

Merancang arsitektur dan desain sistem secara keseluruhan.

3) *Implementation*

Mengkodekan sistem berdasarkan desain yang telah dibuat.

4) *Integration & Testing*

Mengintegrasikan komponen dan melakukan pengujian sistem.

5) *Deployment*

Menempatkan sistem ke lingkungan produksi.

6) *Maintenance*

Pemeliharaan dan perbaikan sistem setelah *deployment*.

3.6 *Unified Modeling Language (UML)*

Menurut Fowler (2004), *Unified Modeling Language (UML)* adalah bahasa pemodelan standar yang digunakan untuk visualisasi, spesifikasi, konstruksi, dan dokumentasi dari artifak sistem perangkat lunak. UML menyediakan cara standar untuk menulis *blueprint* sistem, mencakup aspek konseptual seperti proses bisnis dan fungsi sistem.

Booch et al. (2005) menjelaskan bahwa UML adalah bahasa visual untuk pemodelan yang dapat digunakan untuk :

- 1) Visualisasi struktur dan perilaku sistem
- 2) Komunikasi antar *stakeholder* proyek
- 3) Dokumentasi keputusan desain
- 4) *Template* untuk konstruksi sistem

3.6.1 Jenis Diagram UML

Menurut Dennis et al. (2015), UML menyediakan berbagai jenis diagram yang dapat dikategorikan menjadi :

1) *Structural Diagrams*

1. *Class Diagram*

Class Diagram digunakan untuk menggambarkan struktur statis dari suatu sistem, yaitu bagaimana kelas-kelas (*classes*),

atribut, metode (*operations*), dan hubungan antar kelas disusun dalam sistem perangkat lunak.

2. *Object Diagram*

Object Diagram menunjukkan snapshot atau kondisi konkret dari objek dalam sistem pada suatu titik waktu tertentu, berdasarkan struktur yang didefinisikan oleh *Class Diagram*.

3. *Component Diagram*

Component Diagram menggambarkan organisasi dan ketergantungan antar komponen perangkat lunak, yang merepresentasikan bagian fisik dari sistem seperti file, modul, atau layanan.

2) *Behavioral Diagrams*

1. *Use Case Diagram*

Menunjukkan interaksi antara aktor dan sistem.

2. *Activity Diagram*

Menunjukkan alur kerja atau proses bisnis.

3. *Sequence Diagram*

Menunjukkan interaksi antar objek dalam urutan waktu.

3.6.2 Use Case Diagram

Menurut Cockburn (2001), *Use Case Diagram* adalah representasi grafis dari interaksi antara pengguna (aktor) dengan sistem untuk mencapai tujuan tertentu. *Use case* mendeskripsikan fungsionalitas sistem dari perspektif pengguna.

Larman (2005) menjelaskan komponen utama *use case diagram* :

1) *Actor*

Entitas eksternal yang berinteraksi dengan sistem.

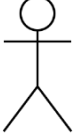
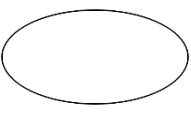
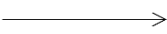
2) *Use Case*

Fungsi atau layanan yang disediakan sistem.

3) *Association*

Hubungan antara *actor* dan *use case*.

Tabel 3. 1 Simbol Use Case Diagram (Cockburn, 2001)

No	Nama	Simbol	Keterangan
1.	<i>Actor</i>		Menggambarkan orang lain atau sistem yang menyediakan atau menerima informasi dari sistem atau menggambarkan pengguna <i>software</i> aplikasi (<i>user</i>).
2.	<i>Use Case</i>		Menggambarkan fungsionalitas dari suatu sistem, sehingga pengguna sistem paham dan mengerti mengenai kegunaan sistem yang akan dibangun.
3.	<i>Association</i>		Menggambarkan hubungan antara <i>actor</i> dengan <i>use case</i> .

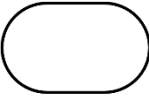
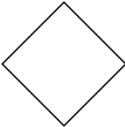
3.6.3 Activity Diagram

Menurut Rumbaugh et al. (2005), Activity Diagram adalah diagram yang menunjukkan alur kerja (workflow) dari sebuah proses bisnis atau use case. Diagram ini berguna untuk memodelkan proses yang kompleks dan menunjukkan logika kondisional.

Komponen utama activity diagram menurut Ambler (2005) :

- 1) *Start Node* (Titik awal aktivitas)
- 2) *Activity* (Tugas atau aksi yang dilakukan)
- 3) *Decision Node* (Titik keputusan dengan *multiple path*)
- 4) *End Node* (Titik akhir aktivitas)

Tabel 3. 2 Simbol Activity Diagram (Rumbaugh et al., 2005)

No	Nama	Simbol	Keterangan
1.	<i>Start Node</i>		Titik awal dari alur aktivitas yang menandakan dimulainya suatu proses atau tugas.
2.	<i>Activity</i>		Menunjukkan tugas atau langkah yang dilakukan dalam proses, seperti tindakan atau pekerjaan yang perlu diselesaikan.
3.	<i>Decision Node</i>		Titik percabangan yang memungkinkan alur mengambil jalur berbeda berdasarkan kondisi yang ditentukan.
4.	<i>End Node</i>		Titik akhir dari alur aktivitas yang menandakan proses atau tugas selesai.

3.6.4 Sequence Diagram

Berdasarkan Pilone dan Pitman (2005), *Sequence Diagram* adalah diagram yang menunjukkan interaksi antar objek dalam urutan waktu yang spesifik. Diagram ini berguna untuk menunjukkan bagaimana objek berkolaborasi untuk mencapai fungsionalitas tertentu.

Komponen *sequence diagram* meliputi :

1) *Lifeline*

Lifeline menggambarkan keberadaan dari sebuah objek atau aktor selama proses interaksi berlangsung dalam *sequence diagram*.

2) *Activation Box*

Activation Box atau *activation bar* adalah blok persegi panjang vertikal kecil di atas *lifeline* yang menunjukkan kapan objek sedang aktif atau melaksanakan suatu operasi.

3) *Message*

Message adalah panah horizontal dari satu *lifeline* ke *lifeline* lain yang menunjukkan pengiriman pesan (*method call* atau perintah).

4) *Return Message*

Return Message adalah panah balik dari objek penerima ke objek pengirim yang menandakan bahwa objek penerima telah menyelesaikan operasi dan mengirim hasilnya kembali.

Tabel 3. 3 Simbol Sequence Diagram (Pilone dan Pitman, 2005)

No	Nama	Simbol	Keterangan
1.	<i>Lifeline</i>		Menunjukkan entitas atau objek yang terlibat dalam interaksi.
2.	<i>Activation Bar</i>		Menunjukkan periode waktu di mana objek atau entitas aktif dan melakukan suatu tindakan atau proses.
3.	<i>Message</i>		Menunjukkan komunikasi atau pengiriman informasi antara objek.
4.	<i>Return Message</i>		Mengindikasikan balasan atau respon yang diterima setelah pengiriman pesan.

3.7 Desain Antar Muka

Menurut Sharp et al. (2019), desain antarmuka sistem adalah proses perancangan interaksi antara pengguna dan sistem komputer yang berfokus pada pembuatan antarmuka yang efektif, efisien, dan memberikan pengalaman pengguna yang baik. Desain antarmuka mencakup tata letak visual, navigasi, dan interaksi pengguna dengan sistem.

Cooper et al. (2014) menjelaskan bahwa desain antarmuka yang baik harus mempertimbangkan prinsip-prinsip *usabilitas*, yaitu *learnability* (mudah dipelajari), *efficiency* (efisien dalam penggunaan), *memorability* (mudah

diingat), *error handling* (penanganan kesalahan yang baik), dan *satisfaction* (kepuasan pengguna).

3.7.1 *Wireframe*

Menurut Garrett (2011), *wireframe* adalah representasi visual sederhana dari antarmuka (*user interface*) suatu aplikasi atau situs web yang digunakan untuk merancang tata letak dan struktur halaman. *Wireframe* berfungsi sebagai *blueprint* atau kerangka dasar yang menunjukkan posisi elemen-elemen antarmuka tanpa detail visual seperti warna, gambar, atau tipografi.

Brown (2013) mendefinisikan *wireframe* sebagai diagram skematis yang menggambarkan struktur dan fungsionalitas halaman web atau aplikasi. Wireframe membantu desainer dan pengembang untuk :

- 1) Perencanaan Struktur

Menentukan hierarki informasi dan tata letak elemen-elemen pada halaman.

- 2) Komunikasi Konsep

Menyampaikan ide dan konsep desain kepada stakeholder tanpa terganggu oleh detail visual.

- 3) Identifikasi Masalah

Mengidentifikasi potensi masalah navigasi atau alur kerja sebelum tahap pengembangan.

- 4) Efisiensi Pengembangan

Menghemat waktu dan biaya dengan menentukan struktur sebelum implementasi.



Gambar 3. 1 Wireframe (Garrett, 2011 dan Brown, 2013)

3.8 Administrasi Pendidikan

Menurut Sagala (2013), administrasi pendidikan adalah keseluruhan proses kerjasama dengan memanfaatkan semua sumber personil dan materil yang tersedia dan sesuai untuk mencapai tujuan pendidikan yang telah ditetapkan secara efektif dan efisien.

Purwanto (2014) mendefinisikan administrasi pendidikan sebagai suatu ilmu yang mempelajari penataan sumber daya untuk mencapai tujuan pendidikan secara produktif. Administrasi pendidikan mencakup perencanaan, pengorganisasian, pengarahan, pengkoordinasian, pengawasan, dan evaluasi terhadap kegiatan-kegiatan pendidikan.

3.8.1 Ruang Lingkup Administrasi Pendidikan

Menurut Mulyasa (2007), ruang lingkup administrasi pendidikan meliputi:

1) Administrasi Kurikulum

Pengelolaan kurikulum dan proses pembelajaran

2) Administrasi Kesiswaan

Pengelolaan data dan aktivitas siswa

3) Administrasi Kepegawaian

Pengelolaan data dan aktivitas tenaga pendidik

4) Administrasi Keuangan

Pengelolaan sumber daya finansial

5) Administrasi Sarana Prasarana

Pengelolaan fasilitas pendidikan

3.8.2 Administrasi Guru

Menurut Daryanto (2013), administrasi guru adalah kegiatan pencatatan data dan informasi tentang guru dalam rangka pelaksanaan program sekolah. Administrasi guru meliputi data pribadi guru, riwayat pendidikan, pengalaman mengajar, dan dokumen-dokumen pembelajaran.

Suhandan et al. (2012) menjelaskan bahwa administrasi guru mencakup berbagai komponen penting dalam pengelolaan data dan

aktivitas guru, termasuk perencanaan pembelajaran, pelaksanaan pembelajaran, evaluasi, dan pengembangan profesional guru.

Dalam konteks Kurikulum 2013, administrasi guru terbagi dalam empat kelompok buku kerja yang terstandar. Menurut Kemendikbud (2017), pembagian buku kerja guru meliputi:

1) Buku Kerja Guru 1

Capaian Pembelajaran (CP), Tujuan Pembelajaran (TP), Alur Tujuan Pembelajaran (ATP), Modul Pembelajaran, Kriteria Ketercapaian Tujuan Pembelajaran, Program Remedial Pengayaan.

2) Buku Kerja Guru 2

Kode Etik Guru, Ikrar Guru, Tata Tertib, Pembiasaan Guru, Kalender Pendidikan, Analisis Alokasi Waktu, Program Tahunan, Program Semester, Jurnal Agenda Guru.

3) Buku Kerja Guru 3

Daftar Hadir Siswa, Daftar Nilai, Penilaian Kepribadian Siswa, Analisis Hasil Penilaian, Program Remedial dan Pengayaan, Daftar Buku Guru dan Siswa, Jadwal Mengajar, Daya Serap Siswa, Kumpulan Kisi Kisi Soal, Kumpulan Soal, Analisis Butir Soal, Perbaikan Soal.

4) Buku Kerja Guru 4

Daftar Evaluasi Diri Kerja Guru dan Program Tindak Lanjut Kinerja.

3.9 Metodologi Pengujian *Black Box*

Menurut Myers et al. (2012), *Black Box Testing* adalah metodologi pengujian perangkat lunak yang berfokus pada pengujian fungsionalitas aplikasi tanpa mengetahui struktur internal, desain, atau implementasi dari item yang

sedang diuji. Pengujian ini hanya mempertimbangkan input dan output yang diharapkan.

Sommerville (2016) menjelaskan bahwa *black box testing* adalah teknik pengujian dimana *tester* tidak memiliki akses terhadap *source code* dan struktur internal sistem. Pengujian dilakukan berdasarkan spesifikasi *requirement* dan fungsionalitas yang diharapkan.

3.8.3 Teknik Black Box Testing

Menurut Jorgensen (2013), terdapat beberapa teknik dalam *black box testing* :

1) *Equivalence Partitioning*

Membagi *input domain* menjadi kelas ekuivalen

2) *Boundary Value Analysis*

Menguji nilai-nilai pada batas *domain input*.

3) *Decision Table Testing*

Menggunakan tabel keputusan untuk *test case*.

4) *State Transition Testing*

Menguji perubahan *state* dalam sistem.

3.8.4 Keunggulan dan Kelemahan *Black Box Testing*

Khan dan Khan (2012) mengidentifikasi keunggulan *black box testing* :

- 1) Tidak memerlukan pengetahuan teknis tentang programming
- 2) Efisien untuk sistem yang besar dan kompleks
- 3) Menguji dari perspektif end-user
- 4) Dapat mengidentifikasi missing functionality

Kelemahan *black box testing* menurut Patton (2006) :

- 1) *Coverage testing* terbatas.

- 2) Tidak dapat mengidentifikasi *unused code*.
- 3) *Test case design* dapat menjadi kompleks untuk input yang besar.
- 4) Kemungkinan redundansi dalam *test cases*.